Getting Started with EF Core

Overview



Introduction to EF Core

EF Core Version History

EF Core Development Approaches

EF Core vs EF6

EF Core Database Providers

Introduction to EF Core

- Entity Framework Core is the new version of Entity Framework after EF 6.x. It is open-source, lightweight, extensible and a cross-platform version of Entity Framework data access technology.

- Entity Framework is an **Object/Relational Mapping (O/RM) framework**. It is an enhancement to ADO.NET that gives developers an automated mechanism for accessing & storing the data in the database.

- EF Core is intended to be used with .NET Core applications. However, it can also be used with standard .NET 4.5+ framework based applications.

- **What is Object Relational Mapping?**

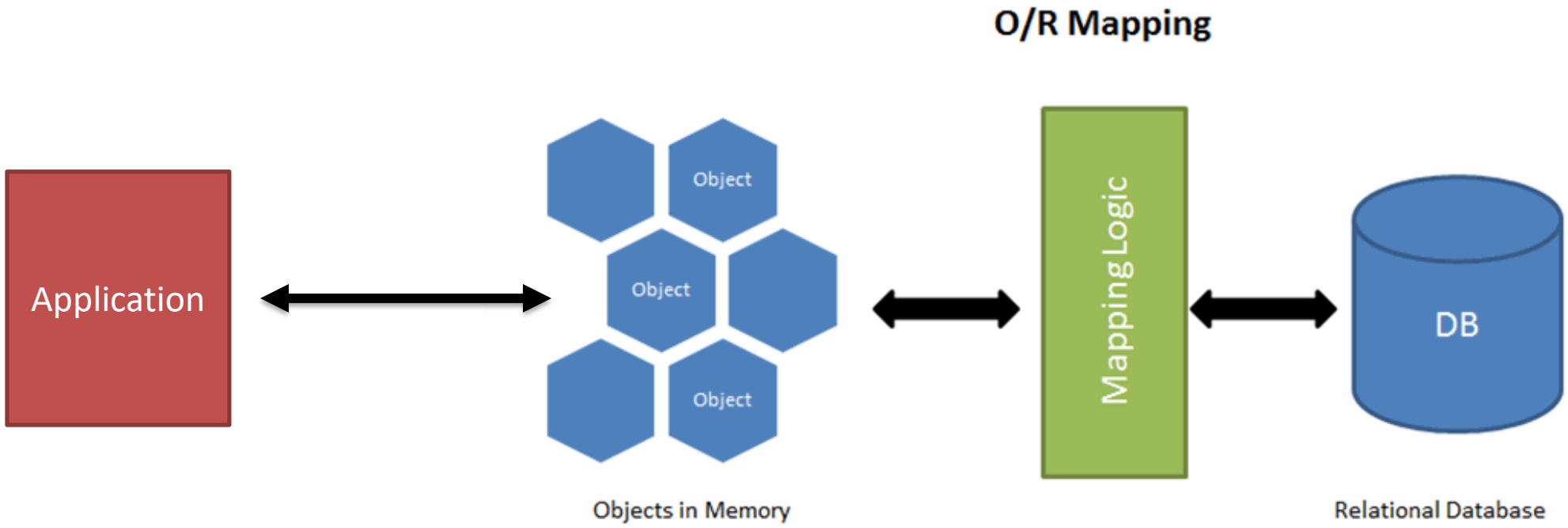
ORM (Object-Relational Mapping) is a programming technique and a framework that facilitates the interaction between object-oriented programming languages and databases.

An ORM framework acts as a bridge between the application's object model and the underlying database, allowing developers to work with database entities as objects and abstracting away the complexities of SQL queries and database interactions.

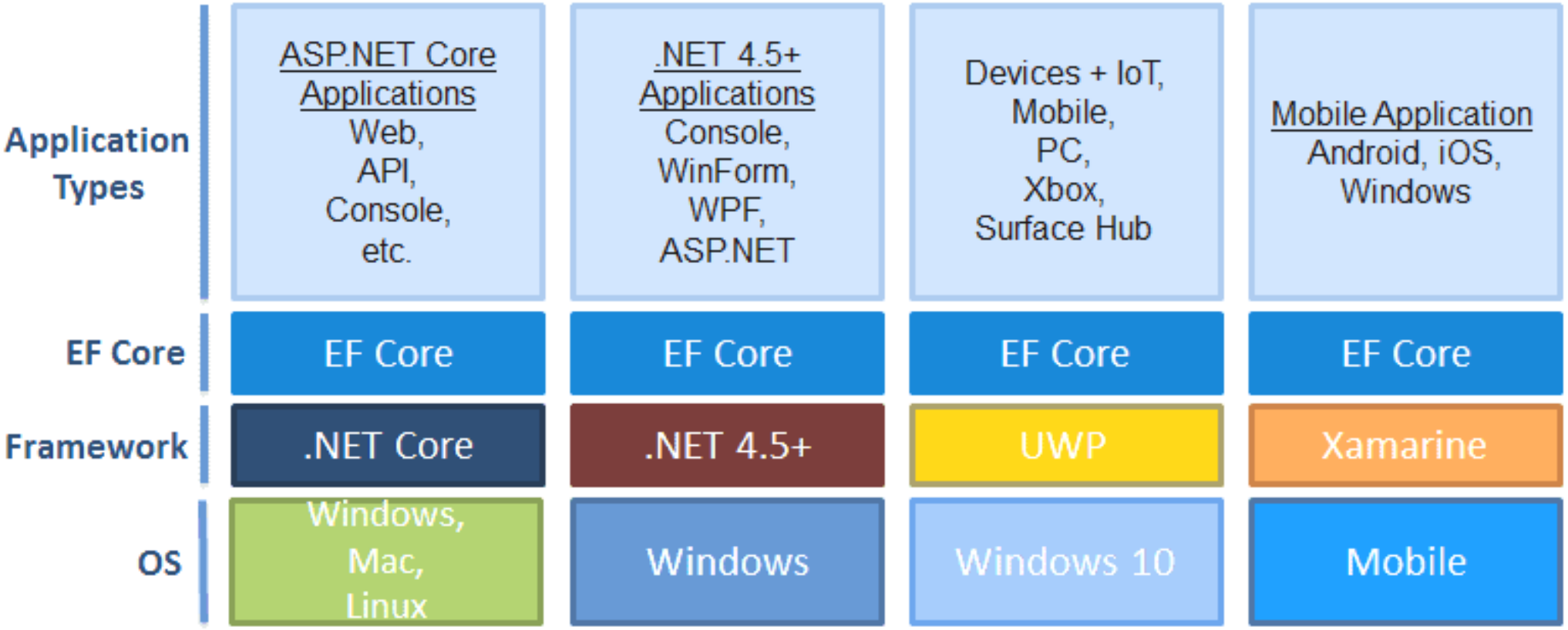
Some popular ORM frameworks include Entity Framework (EF), Hibernate (for Java), Django ORM (for Python), and Laravel Eloquent (for PHP).

Overall, ORM frameworks simplify the development process by bridging the gap between object-oriented code and relational databases, providing increased productivity,

Object Relational Mapping



The following figure illustrates the supported application types, .NET Frameworks and OSs.



EF Core Version History

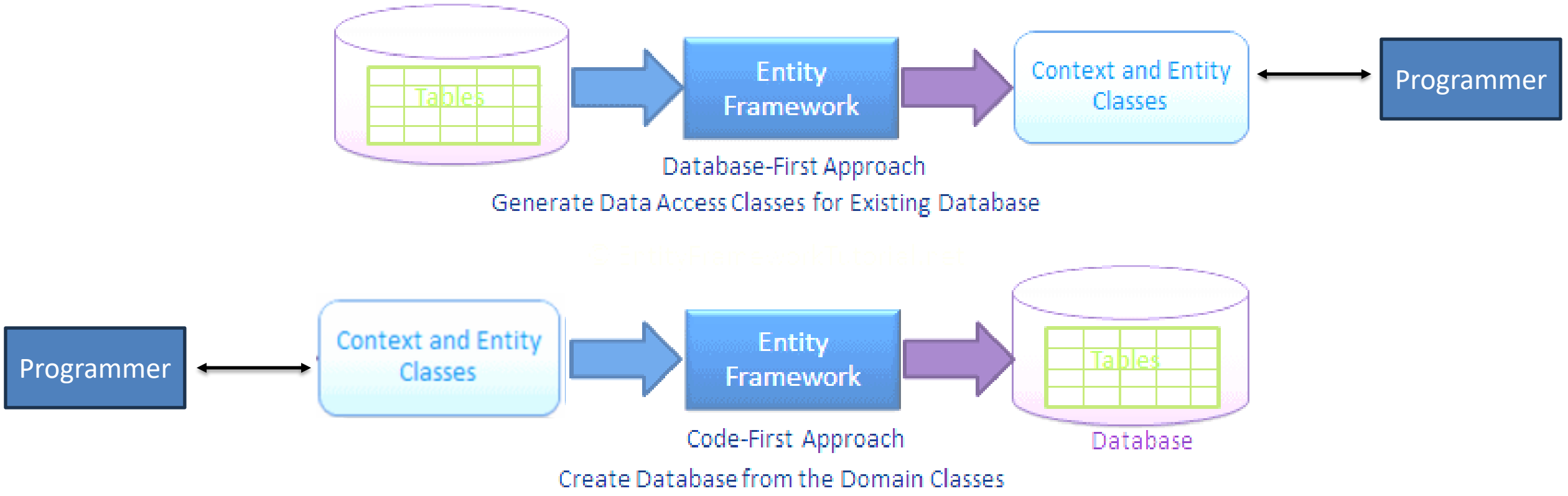
EF Core Version History

Version	Release Date	.NET Version	Key Highlights
EF Core 1.0	June 2016	.NET Core 1.0	Initial release (cross-platform ORM)
EF Core 2.0	Aug 2017	.NET Core 2.0	Improved performance, LINQ enhancements
EF Core 3.0	Sept 2019	.NET Core 3.0	Breaking changes, better LINQ translation
EF Core 3.1 (LTS)	Dec 2019	.NET Core 3.1	Long-Term Support version
EF Core 5.0	Nov 2020	.NET 5	Many-to-many, improved queries
EF Core 6.0 (LTS)	Nov 2021	.NET 6	Performance boost, minimal APIs support
EF Core 7.0	Nov 2022	.NET 6/7	JSON columns, bulk updates
EF Core 8.0 (LTS)	Nov 2023	.NET 8	Complex types, primitive collections
EF Core 9.0	Nov 2024	.NET 8	Incremental improvements, performance
EF Core 10.0 (LTS)	Nov 2025	.NET 10	Latest stable, long-term support till 2028

EF Core Development Approaches

EF Core Development Approaches

- EF Core supports two development approaches ;Code-First, Database-First.
- EF Core mainly targets the code-first approach and provides little support for the database-first approach.



Differences Between DB First and Code First Approach

DB First	Code First
Database Design First: The database schema is designed and created first using tools such as SQL Server Management Studio or other database design tools.	**Code Design First:** The developer designs the domain model classes and relationships in code without considering the database schema.
Automatic Code Generation: Once the database schema is in place, EF generates the corresponding entity classes, relationships, and DbContext based on the existing database.	**Automatic Database Generation:** Based on the code-defined models and configurations, EF creates the corresponding database schema automatically during runtime or through migration scripts.
Design Changes Impact Database: Any changes to the database schema require updating the code manually or using tools to regenerate the code.	**Code-Driven Migrations: E**nables seamless migrations where developers can modify the code-first model and use EF migrations to update the database schema accordingly.
Existing Database Support: DB-first is commonly used when working with an existing database.	**Database-Agnostic:** Allows working with different database providers (e.g., SQL Server, MySQL, SQLite) without directly dealing with the database-specific implementation details.
	Development Speed and Agility: Facilitates rapid development.

- **Choosing the appropriate approach in Entity Framework Core.**

Project Requirements: Understand the specific requirements of your project. Consider factors such as the complexity of the domain model, the need for rapid development or prototyping, the existence of an already established database, and the level of control and customization required.

Existing Infrastructure: If you are working with an existing database, the DB-first approach may be more suitable as it allows you to map the database schema to code. On the other hand, if you are starting from scratch or have more flexibility in designing the database, the Code-first approach may provide more convenience and productivity.

Development Lifecycle: Consider the development lifecycle and how the database and code will evolve over time. If your project requires frequent database schema changes or iterative development, the Code-first approach with migrations support can be beneficial. If the database schema is relatively stable and won't change frequently, the DB-first approach might be more appropriate.

Team Skills and Preferences: Evaluate the skills and preferences of your development team. If your team is experienced with database design and prefers working with SQL or database tools, the DB-first approach may be a better fit. Conversely, if your team prefers a code-centric approach and wants more control over the domain model and relationships, the Code-first approach might be more appealing.

Database Provider Flexibility: Consider whether your project needs to support multiple database providers or if you have a specific database provider requirement. The Code-first approach provides flexibility to switch between different database providers easily, while the DB-first approach is more tied to a specific database schema.

EF Core Database Provider

EF Core Database Providers

Entity Framework Core uses a provider model to access many different databases. EF Core includes providers as NuGet packages which you need to install.

Database	NuGet Package
SQL Server	Microsoft.EntityFrameworkCore.SqlServer
MySQL	MySql.Data.EntityFrameworkCore
PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL
SQLite	Microsoft.EntityFrameworkCore.SQLite
SQL Compact	EntityFrameworkCore.SqlServerCompact40
In-memory	Microsoft.EntityFrameworkCore.InMemory

References

https://learn.microsoft.com/en-us/ef/core

https://dotnettutorials.net/lesson/entity-framework-core/

https://www.entityframeworktutorial.net/efcore/entity-framework-core.aspx

https://www.c-sharpcorner.com/article/introduction-to-entity-framework-core/

https://codepedia.info/ef-core/entity-framework-core-introduction-5

https://www.learnentityframeworkcore.com/

https://www.devart.com/what-is-entity-framework-core/